

1. Print the boundary nodes of a binary tree.

```
2. class Solution {
3.     void left(Node root,ArrayList<Integer> ans){
4.         if(root==null) return;
5.         if(root.left!=null){
6.             ans.add(root.data);
7.             left(root.left,ans);
8.         }else if(root.right!=null){
9.             ans.add(root.data);
10.            left(root.right,ans);
11.        }
12.    }
13.    void right(Node root,ArrayList<Integer> ans){
14.        if(root==null) return;
15.        if(root.right!=null){
16.            right(root.right,ans);
17.            ans.add(root.data);
18.        }
19.        else if(root.left!=null){
20.            right(root.left,ans);
21.            ans.add(root.data);
22.        }
23.    }
24.    void leaf(Node root,ArrayList<Integer> ans){
25.        if(root==null) return;
26.        if(root.left==null&&root.right==null)ans.add(root.data);
27.        leaf(root.left,ans);
28.        leaf(root.right,ans);
29.    }
30.
31.    ArrayList<Integer> boundaryTraversal(Node root) {
32.        ArrayList<Integer> ans = new ArrayList<>();
33.        ans.add(root.data);
34.        left(root.left,ans);
35.        leaf(root.left,ans);
36.        leaf(root.right,ans);
37.        right(root.right,ans);
38.        return ans;
39.    }
40. }
```

2. Print the left view of a binary tree.

```
class Solution {
    public ArrayList<Integer> leftView(Node root) {
        ArrayList<ArrayList<Integer>> ans = new ArrayList<>();
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        while(!q.isEmpty()){
            int size = q.size();
```

```

        ArrayList<Integer> al = new ArrayList<>();
        for(int i=0;i<size;i++){
            Node cur = q.poll();
            if(cur!=null){
                if(cur.left!=null) q.add(cur.left);
                if(cur.right!=null) q.add(cur.right);
            }al.add(cur.data);
        }ans.add(al);
    }ArrayList<Integer> al = new ArrayList<>();
    for(int i=0;i<ans.size();i++){
        al.add(ans.get(i).get(0));
    }return al;
}
}

```

3. Print the right view of a binary tree.

```

class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        Queue<TreeNode> q = new LinkedList<>();
        q.add(root);
        while(!q.isEmpty()){
            int n = q.size();
            List<Integer> al = new ArrayList<>();
            for(int i=0;i<n;i++){
                TreeNode curr = q.poll();
                if(curr!=null){
                    if(curr.left!=null)q.add(curr.left);
                    if(curr.right!=null)q.add(curr.right);
                    al.add(curr.val);
                }
            }if (!al.isEmpty()) {
                ans.add(al);
            }
        }List<Integer> al = new ArrayList<>();
        for(int i=0;i<ans.size();i++){
            al.add(ans.get(i).get(ans.get(i).size()-1));
        }return al;
    }
}

```

4. Find the lowest common ancestor (LCA) of two nodes in a binary tree.

```

class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root==null) return null;
        if(root==p||root==q)
            return root;
        TreeNode left = lowestCommonAncestor(root.left,p,q);
    }
}

```

```

        TreeNode right = lowestCommonAncestor(root.right,p,q);
        if(left !=null && right !=null) return root;
        if(left==null) return right;
        return left;
    }
}

```

5. Find the path from the root to a given node in a binary tree.

```

class Solution {
    public boolean findPath(TreeNode root, int target, List<Integer> path) {
        if (root == null) {
            return false;
        }
        path.add(root.val);
        if (root.val == target) {
            return true;
        }
        if (findPath(root.left, target, path) ||
            findPath(root.right, target, path)) {
            return true;
        }
        path.remove(path.size() - 1);
        return false;
    }
}

```

```

class Solution {
    int i =0;
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        HashMap<Integer,Integer> inMap = new HashMap<>();
        for(int i=0;i<inorder.length;i++)
            inMap.put(inorder[i],i);
        return constructTree(preorder,inorder,inMap,0,inorder.length-1);
    }
    TreeNode constructTree(int[] pre,int[] in,HashMap<Integer,Integer> inMap,int
start,int end){
        if(start>end)
            return null;
        TreeNode nn = new TreeNode(pre[i]);
        int idx = inMap.get(pre[i]);
        i++;
        nn.left = constructTree(pre,in,inMap,start,idx-1);
        nn.right = constructTree(pre,in,inMap,idx+1,end);
        return nn;
    }
}

```